BFS DFS TRÊN MA TRẬN VÀ ĐỒ THỊ

Quy ước mê cung: `.` là ô có thể đi, `#` là tường. Xuất phát từ `(startX, startY)`, các bước chuyển được quy định bằng `dx, dy`. Có thể xét 4 ô kề cạnh, 8 ô xung quanh hoặc bước đi tùy đề. Hàng, cột và đỉnh đều đánh số từ 1.

1. Input và khởi tạo mê cung char[][]

```
import java.io.*;
import java.util.*;

static int N, M;
static char[][] maze;
static boolean[][] visited;
static int[] dx = {-1, 1, 0, 0}; // vi du: 4 huong
static int[] dy = {0, 0, -1, 1};

// Goi trong main (co the nem IOException):
BufferedReader br = new BufferedReader(
        new InputStreamReader(System.in));
StringTokenizer st = new StringTokenizer(br.readLine());
N = Integer.parseInt(st.nextToken());
M = Integer.parseInt(st.nextToken());
maze = new char[N + 1][M + 1];
visited = new boolean[N + 1][M + 1];
for (int i = 1; i <= N; i++) {
    String line = br.readLine().trim();
    for (int j = 1; j <= M; j++) {
        maze[i][j] = line.charAt(j - 1);
    }
}
```

Ví dụ input (4 dòng mê cung, mỗi dòng đúng M ký tự):

```
4 4
...#
.#..
...#
#...
```

2. Hàm kiểm tra ô hợp lệ

```
static boolean canGo(int x, int y) {
    return x >= 1 && x <= N && y >= 1 && y <= M
        && maze[x][y] != '#' && !visited[x][y];
}
```

Ghi nhớ: Mê cung sử dụng `char[][]`. Mảng dx và dy quyết định các bước di chuyển, không phải bản thân BFS hoặc DFS.

3. BFS trên mê cung - Queue

```
static void bfsMaze(int startX, int startY) {
    if (!canGo(startX, startY)) return;
    Queue<int[]> queue = new ArrayDeque<>();
    visited[startX][startY] = true;
    queue.offer(new int[]{startX, startY});

    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        int x = cell[0], y = cell[1];
        System.out.println(x + " " + y);

        for (int d = 0; d < dx.length; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (!canGo(nx, ny)) continue;
            visited[nx][ny] = true; // danh dau khi them
            queue.offer(new int[]{nx, ny});
        }
    }
}
```

4. DFS trên mê cung - Đệ quy

```
static void dfsMaze(int x, int y) {
    if (!canGo(x, y)) return;
    visited[x][y] = true;
    System.out.println(x + " " + y);

    for (int d = 0; d < dx.length; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        dfsMaze(nx, ny); // dau ham se kiem tra o hop le
    }
}
```

Lưu ý: Trước mỗi lần duyệt độc lập, gán lại `visited = new boolean[N + 1][M + 1];`. BFS cho đường đi ngắn nhất theo số bước (mỗi bước có cùng chi phí); DFS không đảm bảo.

MINH HỌA MÊ CUNG VÀ SO SÁNH

5. Ví dụ thứ tự duyệt từ (1,1)

Ví dụ dưới đây dùng 4 hướng theo thứ tự: trên → dưới → trái → phải. Nếu đổi mảng dx/dy thì thứ tự duyệt có thể khác. Mê cung:

.	.	.	#
.	#	.	.
.	.	.	#
#	.	.	.

BFS: (1,1) → (2,1) → (1,2) → (3,1) → (1,3) → (3,2) → (2,3) → (4,2) → (3,3) → (2,4) → (4,3) → (4,4).

DFS: (1,1) → (2,1) → (3,1) → (3,2) → (4,2) → (4,3) → (3,3) → (2,3) → (1,3) → (1,2) → (2,4) → (4,4).

6. So sánh trên mê cung

Tiêu chí	BFS	DFS
Cấu trúc	Queue	Đệ quy / Stack
Cách duyệt	Theo lớp	Đi sâu trước
Đường ngắn nhất	Có (bước cùng chi phí)	Không đảm bảo
Thời gian	O(N × M)	O(N × M)
Bộ nhớ tối đa	O(N × M)	O(N × M)

DFS đệ quy với mê cung rất lớn có thể gây `StackOverflowError`; có thể thay bằng Stack tường minh.

Các kiểu di chuyển trên ma trận

Để xây dựng bảng hướng đi, lấy ô đang xét làm tâm (x,y). Mỗi ô đích có độ lệch `(dx,dy) = (x_moi - x, y_moi - y)`. Trục hàng tăng khi đi xuống, trục cột tăng khi đi sang phải.

4 huong - o ke canh



S: (x,y) o: o co the den

Hang: len = -1; xuong = +1

Cot: trai = -1; phai = +1

```
(-1,0)  (+1,0)
(0,-1)  (0,+1)
```

8 huong - them o ke cheo



S: (x,y) o: o co the den

Hang: len = -1; xuong = +1

Cot: trai = -1; phai = +1

```
(-1,-1) (-1,0) (-1,+1)
(0,-1)           (0,+1)
(+1,-1) (+1,0) (+1,+1)
```

Quan ma - buoc nhay (2,1)



S: (x,y) o: o co the den

Hang: len = -1; xuong = +1

Cot: trai = -1; phai = +1

```
(-2,-1) (-2,+1) (-1,-2) (-1,+2)
(+1,-2) (+1,+2) (+2,-1) (+2,+1)
```

Minh họa: các cặp số trên là `(dx,dy)`, với dx ứng với thay đổi hàng và dy ứng với thay đổi cột. Các phần tử cùng chỉ số trong hai mảng ghép thành một bước di chuyển.

BFS/DFS không mặc định chỉ xét ô kề cạnh. Thay đổi dx/dy tùy cách đề bài định nghĩa hai ô có thể nối với nhau.

```
// 4 o ke canh (tren, duoi, trai, phai)
dx = new int[]{-1, 1, 0, 0};
dy = new int[]{ 0, 0,-1, 1};

// 8 o xung quanh (ke canh + ke cheo)
dx = new int[]{-1,-1,-1, 0, 0, 1, 1, 1};
dy = new int[]{-1, 0, 1,-1, 1,-1, 0, 1};

// Buoc nhay quan ma (8 vi tri)
dx = new int[]{-2,-2,-1,-1, 1, 1, 2, 2};
dy = new int[]{-1, 1,-2, 2,-2, 2,-1, 1};
```

Với bước đi xa hơn một ô, cần đọc kỹ đề: nếu phải đi qua các ô trung gian, hãy kiểm tra cả các ô đó (ví dụ tường chắn). Hàm canGo() hiện chỉ kiểm tra ô đích. Với quy tắc di chuyển phức tạp như trượt đến tường, dịch chuyển tức thời hoặc cạnh nối tùy ý, nên viết hàm sinh các trạng thái tiếp theo thay vì chỉ dùng một mảng dx/dy cố định.

BFS VÀ DFS TRÊN ĐỒ THỊ

Ví dụ dùng đồ thị vô hướng có N đỉnh đánh số 1..N. Lưu bằng danh sách kề `ArrayList<Integer>[]`.

7. Khai báo, tạo danh sách kề và nhập cạnh

```
static int V;
static ArrayList<Integer>[] adj;
static boolean[] seen;

@SuppressWarnings("unchecked")
static void initGraph(int n) {
    V = n;
    adj = new ArrayList[V + 1];
    for (int i = 1; i <= V; i++) adj[i] = new ArrayList<>();
    seen = new boolean[V + 1];
}

static void addEdge(int u, int v) {
    adj[u].add(v);
    adj[v].add(u); // bo dong nay neu do thi co huong
}

// Vi du: initGraph(5); addEdge(1, 2); addEdge(1, 3);
// addEdge(2, 4); addEdge(3, 5);
```

8. BFS trên đồ thị

```
static void bfsGraph(int start) {
    if (start < 1 || start > V) return;
    Queue<Integer> queue = new ArrayDeque<>();
    seen[start] = true;
    queue.offer(start);

    while (!queue.isEmpty()) {
        int u = queue.poll();
        System.out.print(u + " ");
        for (int v : adj[u]) {
            if (seen[v]) continue;
            seen[v] = true;
            queue.offer(v);
        }
    }
}
```

9. DFS trên đồ thị

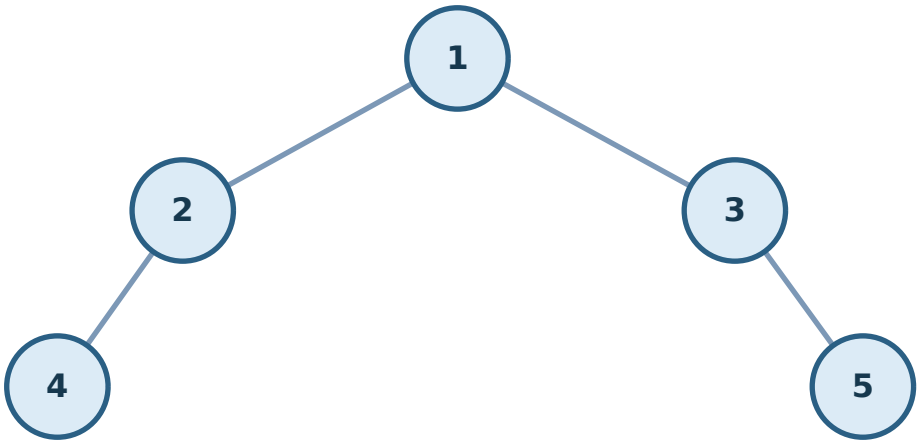
```
static void dfsGraph(int u) {
    if (u < 1 || u > V || seen[u]) return;
    seen[u] = true;
    System.out.print(u + " ");
    for (int v : adj[u]) {
        if (!seen[v]) dfsGraph(v);
    }
}
```

Mỗi lượt duyệt mới cần `seen = new boolean[V + 1];`. Thứ tự duyệt phụ thuộc thứ tự các đỉnh trong danh sách kề.

VÍ DỤ ĐỒ THỊ VÀ GHI NHỚ

10. Đồ thị minh họa

Có 5 đỉnh và các cạnh: (1,2), (1,3), (2,4), (3,5).



BFS từ 1: 1 → 2 → 3 → 4 → 5.

DFS từ 1: 1 → 2 → 4 → 3 → 5.

11. So sánh hai cách biểu diễn

	Mê cung ký tự	**Đồ thị danh sách kề**
Dữ liệu	char[][] maze	ArrayList<Integer>[] adj
Đỉnh	Ô (1,1)..(N,M)	Mỗi đỉnh 1..V
Láng giềng	Tùy bước chuyển dx/dy	adj[u]
Ô/cạnh bị chặn	maze[x][y] == #	Không tồn tại cạnh
Độ phức tạp	O(N × M)	O(V + E)